Data Warehouses

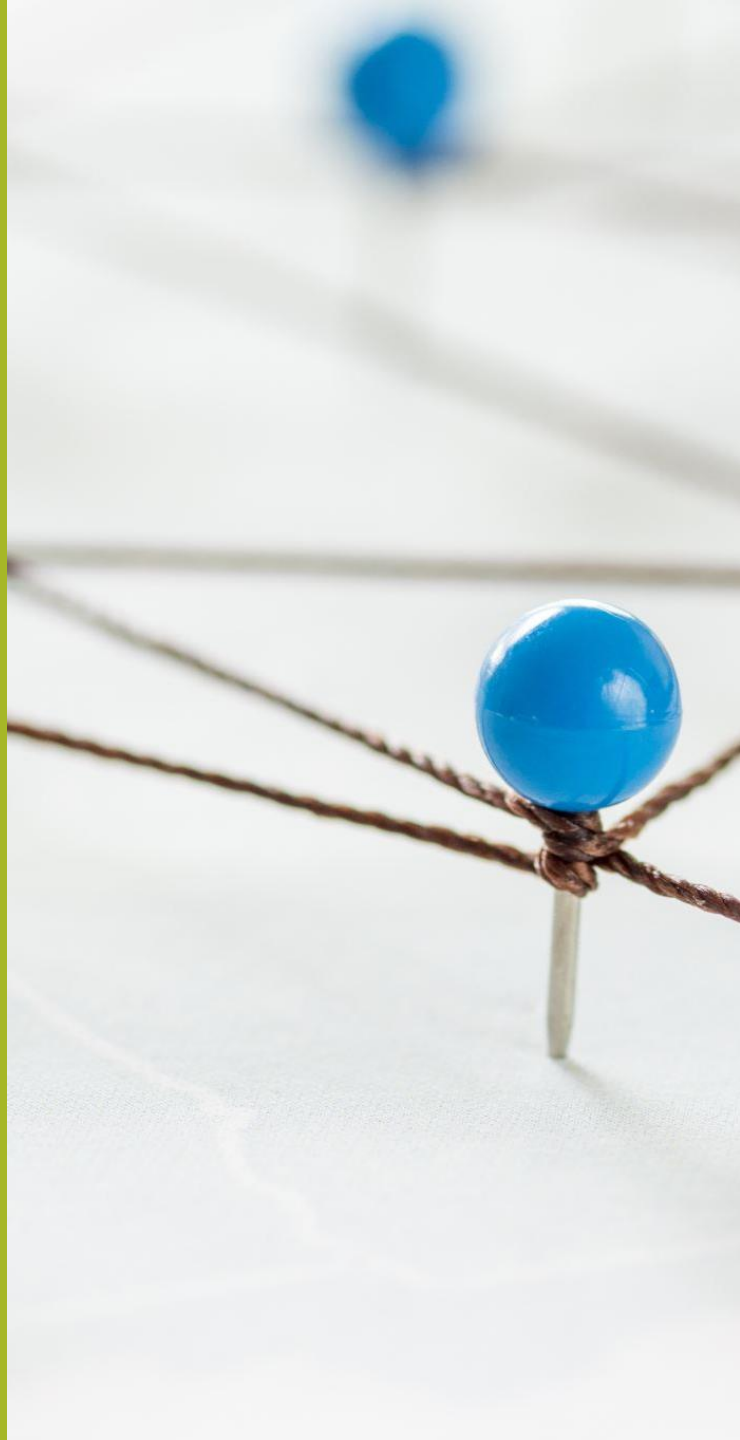
Lecture 9

Lecture Goals

Goal:

- Logical Data Map
- Handling changes – SCDs
  - *Intro, types & usage*

Logical Data Map



Logical Map

Before you begin building your extract systems,

you need a logical data map

documents the relationship between original source fields and final destination fields in the tables you deliver to the front room

ties the very beginning of the ETL system to the very end

The physical implementation can be a catastrophe

if it is not carefully architected before it is implemented

just as with any other form of construction

you must have a blueprint before you hit the first nail

Logical Map

Example:

- Information about sales
  - *where is the sale in DB*
- It is a common problem:
  - *that there is no table SALES in the operational databases*
    - Though, some other tables can exist like ORDER with an attribute ORDER STATUS.
  - *that there is no table Time in the operational databases*
    - Though, some tables may contain DATE type attributes

Logical Map

Logical Data Map

- actual design of the logical data mapping document
  - *provide the ETL developer with a clear-cut blueprint of exactly what is expected from the ETL process*
- connects the original source data to the final data
  - *a map for the extraction process*
- contains
  - *the data definition for the data warehouse source systems*
  - *the target data warehouse data model*
  - *the exact manipulation of the data required to transform it from its original format to that of its final destination.*

Logical Map

Components of the Logical Data Map

The logical data map is usually presented in a table or spreadsheet format:

Target					Source				Transformation
Table Name	Column Name	Data Type	Table Type	SCD Type	Database Name	Table Name	Column Name	Data Type	
EMPLOYEE_DIM	EMPLOYEE_KEY	NUMBER	Dimension	1				NUMBER	Surrogate key.
EMPLOYEE_DIM	EMPLOYEE_ID	NUMBER	Dimension	1	HR_SYS	EMPLOYEES	EMPLOYEE_ID	NUMBER	Natural Key for employee in HR system
EMPLOYEE_DIM	BIRTH_COUNTRY_NAME	VARCHAR2(75)	Dimension	1	HR_SYS	COUNTRIES	NAME	VARCHAR2(75)	select c.name from employees e, states s, countries c where e.state_id = s.state_id and s.country_id = c.country
EMPLOYEE_DIM	BIRTH_STATE	VARCHAR2(75)	Dimension	1	HR_SYS	STATES	DESCRIPTION	VARCHAR2(255)	select s.description from employees e, states s where e.state_id = s.state_id
EMPLOYEE_DIM	DISPLAY_NAME	VARCHAR2(75)	Dimension	1	HR_SYS	EMPLOYEES	FIRST_NAME	VARCHAR2(75)	select initcap(salutation) ‖ ' '‖initcap(first_name) ‖ ' '‖ initcap(last_name) from employee
EMPLOYEE_DIM	BIRTH_DATE	DATE	Dimension	1	HR_SYS	EMPLOYEES	DOB	DATE	trunc(DOB)
EMPLOYEE_DIM	SALUTATION	VARCHAR2(12)	Dimension	1	HR_SYS	EMPLOYEES	SALUTATION	VARCHAR2(12)	initcap(salutation)
EMPLOYEE_DIM	FIRST_NAME	VARCHAR2(30)	Dimension	1	HR_SYS	EMPLOYEES	FIRST_NAME	VARCHAR2(30)	initcap(first_name)
EMPLOYEE_DIM	LAST_NAME	VARCHAR2(30)	Dimension	1	HR_SYS	EMPLOYEES	LAST_NAME	VARCHAR2(30)	initcap(last_name)
EMPLOYEE_DIM	MARITAL_STATUS	VARCHAR2(12)	Dimension	2	HR_SYS	MARITAL_STATUS	DESCRIPTION	VARCHAR2(12)	select nvl(m.name,'Unknown') from employee e marital_status m where e.marital_status_id = m.marital_status_id
EMPLOYEE_DIM	DIVERSITY_CATEGORY	VARCHAR2(30)	Dimension	1	HR_SYS	EMPLOYEES	EEO_CLASS	VARCHAR2(30)	decode(eeo_class,null, 'Not Stated', decode(eeo_class,'N', 'Not Stated',eeo_class))
EMPLOYEE_DIM	GENDER	VARCHAR2(12)	Dimension	1	HR_SYS	EMPLOYEES	SEX	VARCHAR2(12)	nvl(sex, 'Unknown')
EMPLOYEE_DIM	EMPLOYEE_STATUS	VARCHAR2(24)	Dimension	1	HR_SYS	EMPLOYEES	STATUS	VARCHAR2(24)	select es.name from employee e employee_status es where e.employee_status_id = m.employee_status_id
EMPLOYEE_DIM	POSITION_CODE	VARCHAR2(12)	Dimension	2	HR_SYS	POSITIONS	POSITION_CODE	VARCHAR2(12)	select p.code from employees e, positions p where p.position_id = e.position_id
EMPLOYEE_DIM	POSITION_CATEGORY	VARCHAR2(30)	Dimension	2	HR_SYS	POSITIONS	POSITION_CATEGORY	VARCHAR2(30)	select p.category from employees e, positions p where p.position_id = e.position_id
EMPLOYEE_DIM	HIRE_DATE	DATE	Dimension	1	HR_SYS	EMPLOYEES	DATE_HIRED	DATE	trunc(date_hired)
									select d.code from employee p, employee_department pd, departments d where p.employee_id= pd.employee_id and

Logical Map

Components of the Logical Data Map

The logical data map is usually presented in a table or spreadsheet

Target				
Table Name	Column Name	Data Type	Table Type	SCD Type
EMPLOYEE_DIM	EMPLOYEE_KEY	NUMBER	Dimension	1
EMPLOYEE_DIM	EMPLOYEE_ID	NUMBER	Dimension	1
EMPLOYEE_DIM	BIRTH_COUNTRY_NAME	VARCHAR2(75)	Dimension	1

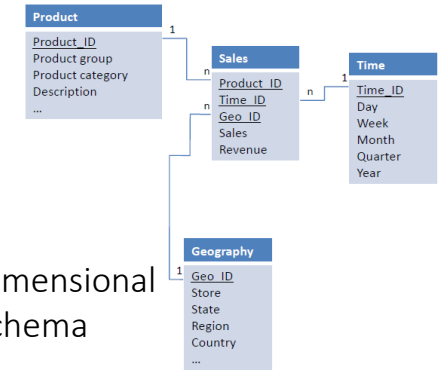
Source			
Database Name	Table Name	Column Name	Data Type
			NUMBER
HR_SYS	EMPLOYEES	EMPLOYEE_ID	NUMBER
HR_SYS	COUNTRIES	NAME	VARCHAR2(75)

Transformation
Surrogate key.
Natural Key for employee in HR system
select c.name from employees e, states s, countries c where e.state_id = s.state_id and s.country_id = c.country
select c.description from employees e, states s

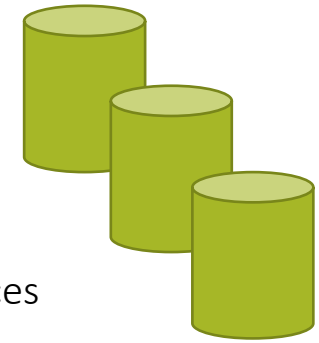
Logical Map

How to prepare Logical Map:

- We start with .. ?
- We finish with .. ?

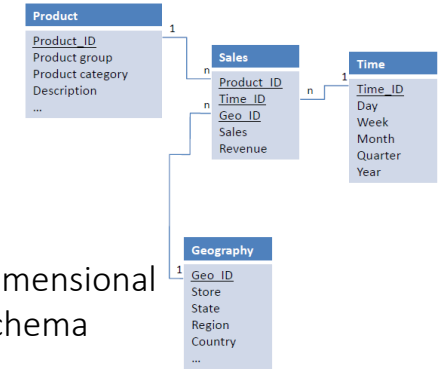


Dimensional Schema



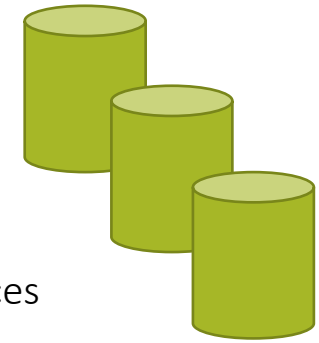
Data Sources

Logical Map



Dimensional Schema

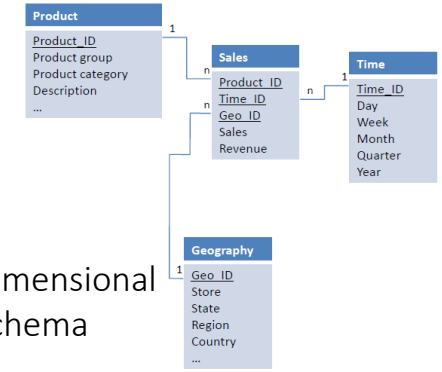
Target				
Table Name	Column Name	Data Type	Table Type	SCD Type
EMPLOYEE_DIM	EMPLOYEE_KEY	NUMBER	Dimension	1
EMPLOYEE_DIM	EMPLOYEE_ID	NUMBER	Dimension	1
EMPLOYEE_DIM	BIRTH_COUNTRY_NAME	VARCHAR2(75)	Dimension	1



Data Sources

Source			
Database Name	Table Name	Column Name	Data Type
			NUMBER
HR_SYS	EMPLOYEES	EMPLOYEE_ID	NUMBER
HR_SYS	COUNTRIES	NAME	VARCHAR2(75)

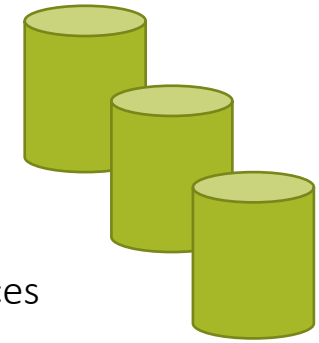
Logical Map



Dimensional Schema

Target				
Table Name	Column Name	Data Type	Table Type	SCD Type
EMPLOYEE_DIM	EMPLOYEE_KEY	NUMBER	Dimension	1
EMPLOYEE_DIM	EMPLOYEE_ID	NUMBER	Dimension	1
EMPLOYEE_DIM	BIRTH_COUNTRY_NAME	VARCHAR2(75)	Dimension	1

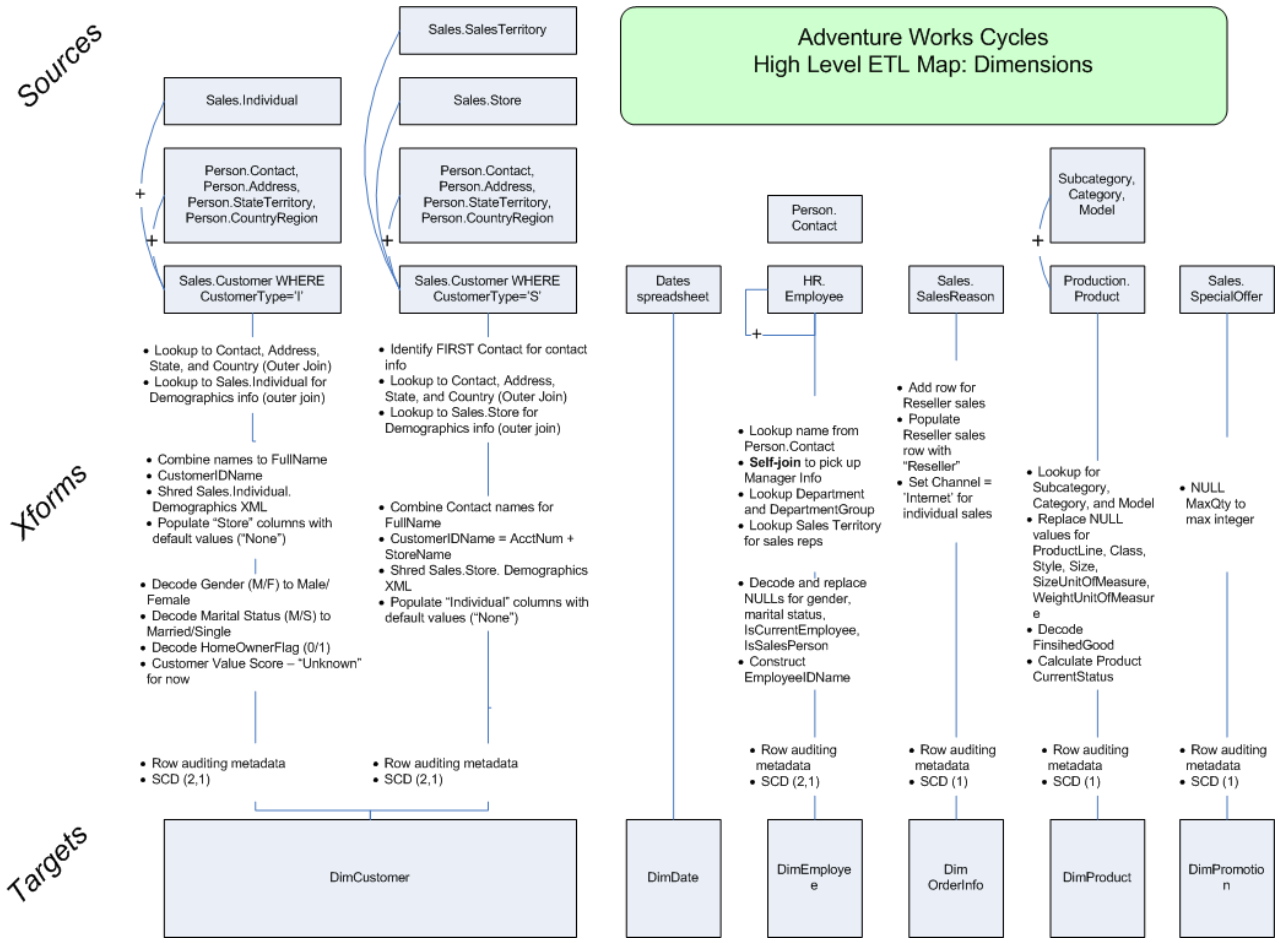
Transformation
Surrogate key.
Natural Key for employee in HR system
select c.name from employees e, states s, countries c where e.state_id = s.state_id and s.country_id = c.country



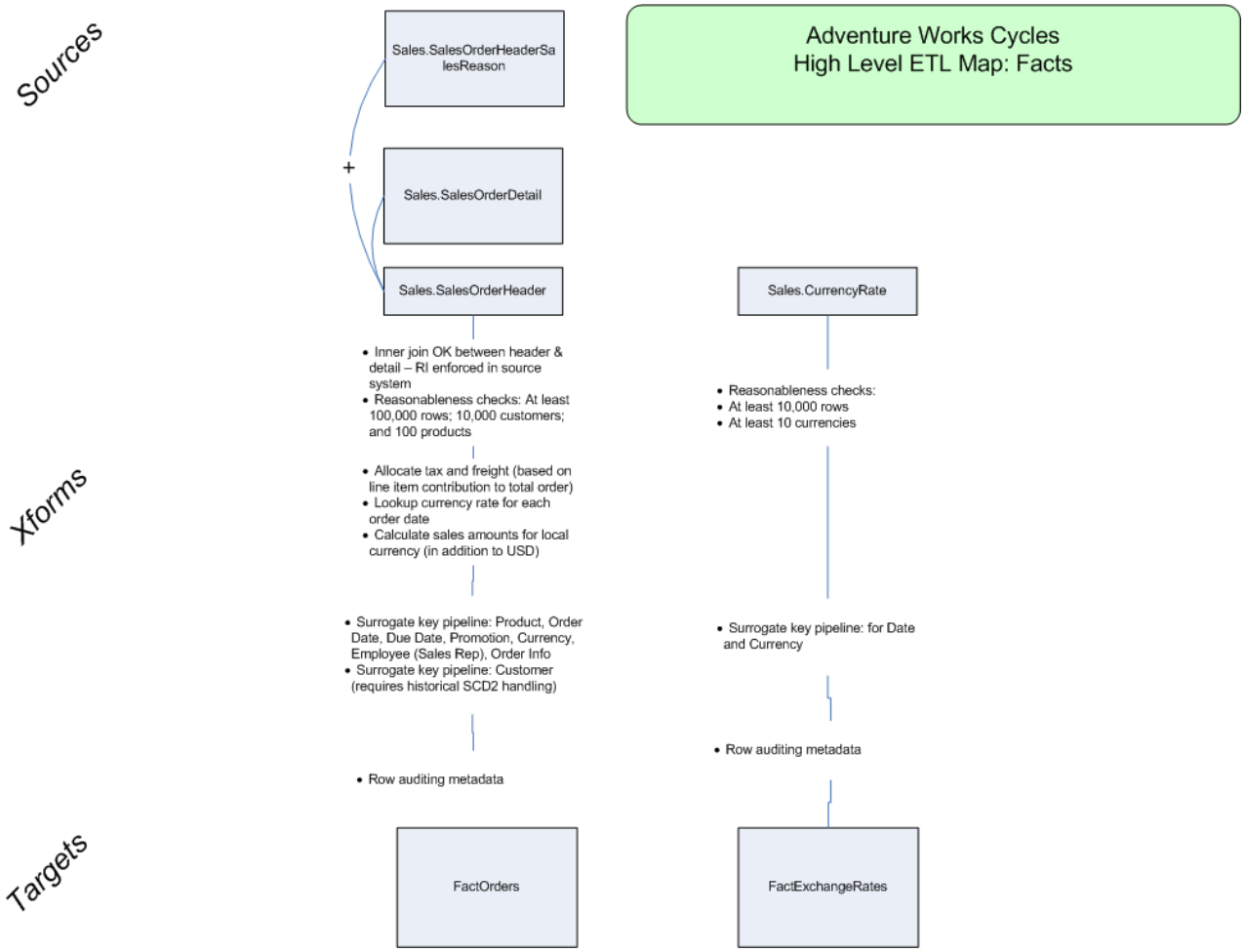
Data Sources

Source			
Database Name	Table Name	Column Name	Data Type
			NUMBER
HR_SYS	EMPLOYEES	EMPLOYEE_ID	NUMBER
HR_SYS	COUNTRIES	NAME	VARCHAR2(75)

High level ETL map



High level ETL map



ETL

Design

Always Logical Before Physical

Have a plan

The ETL process must be figured out logically and documented

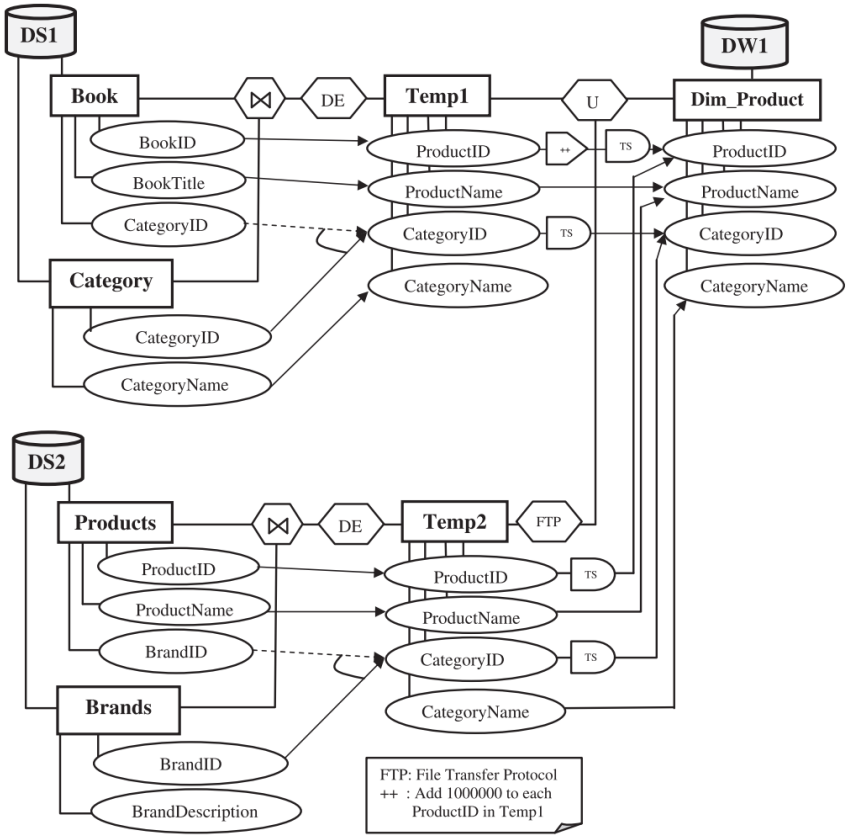


Figure 16 EMD scenario for building products dimension.

ETL

Design (continued):

- Identify data source candidates
  - *Starting with the highest-level business objectives*
    - identify the likely candidate data sources you believe will support the decisions needed by the business community
    - identify specific data elements you believe are central to the end user data
      - These data elements are then the inputs to the data profiling step
- Analyze source systems with a data-profiling tools
  - *Data in the source systems must be scrutinized for data quality and completeness*

ETL

Designing Logical Before Physical (continued):

- Receive walk-though of data lineage and business rules
  - *The data-profiling step should have created two subcategories of ETL-specific business rules:*
    - Required alterations to the data during the data-cleaning steps
    - Coercions to dimensional attributes and measured numerical facts
      - to achieve standard conformance across separate data sources
- Receive walk-through of data warehouse data model
  - *must have a thorough understanding of how dimensions, facts, and other special tables in the dimensional model work together*
- Validate calculations and formulas
  - *Verify with end users any calculations specified in the data linage*
  - *measure twice, cut once*

ETL

Dimensional Data Structures

Dimensional data structures are the target of the ETL processes, and they sit at the boundary between the back room and the front room

In many cases, the dimensional tables will be the final physical-staging step before transferring the tables to the end user environments

- *Fact tables*
- *Dimension tables*
- *Surrogate key mapping tables*

ETL

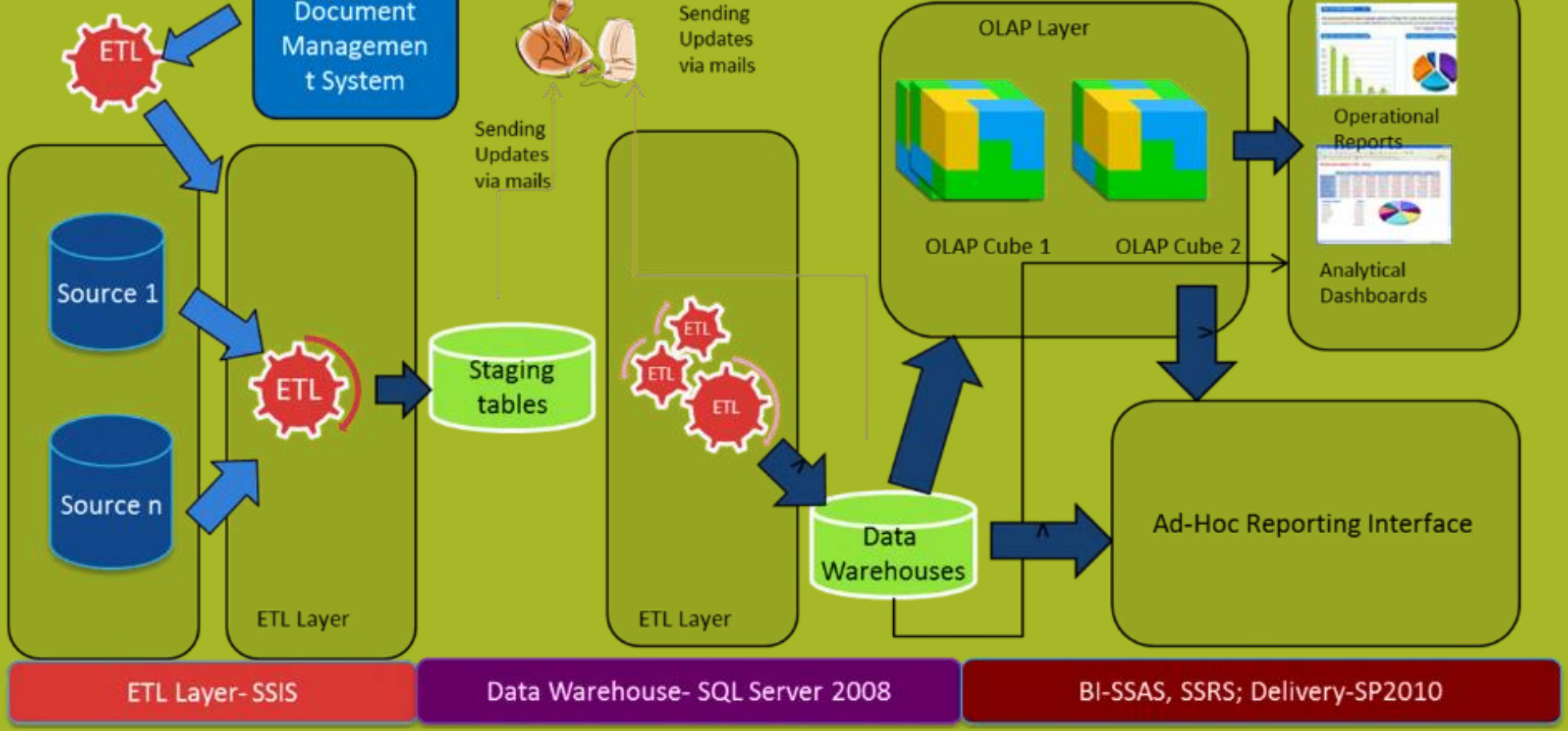
Logical Design First

1. Have a (preliminary) plan
2. Identify data source candidates
3. Analyze source systems with a data-profiling tool

   Possible STOP here
4. Receive walk-through of data lineage and business rules

   Required alterations to the data during the data-cleaning steps

   Coercions to dimensional attributes and measured numerical facts to achieve standard conformance across separate data sources
5. Receive walk-through of data warehouse data model
6. Validate calculations and formulas (*measure twice, cut once*)



Slowly Changing Dimensions

Reality changes - Some information evolves over time

The power of data warehousing stems in part from its ability to provide access to historic data.

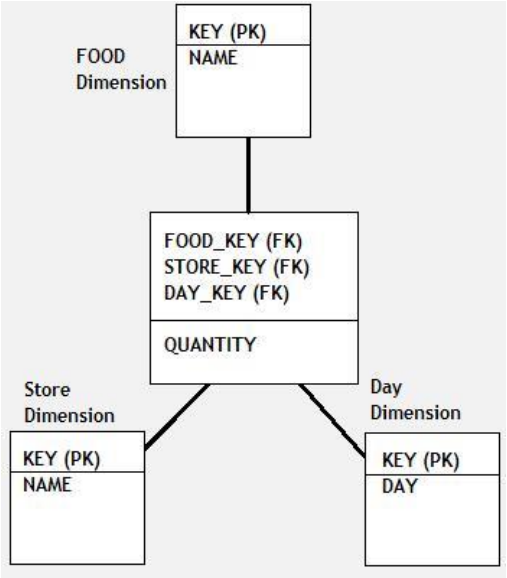
- Data warehouse must be able to respond to changes to information in a way that does not disrupt the ability to study history.
- Dimensional designs deal with this issue through a series of techniques collectively known as “slowly changing dimensions.”

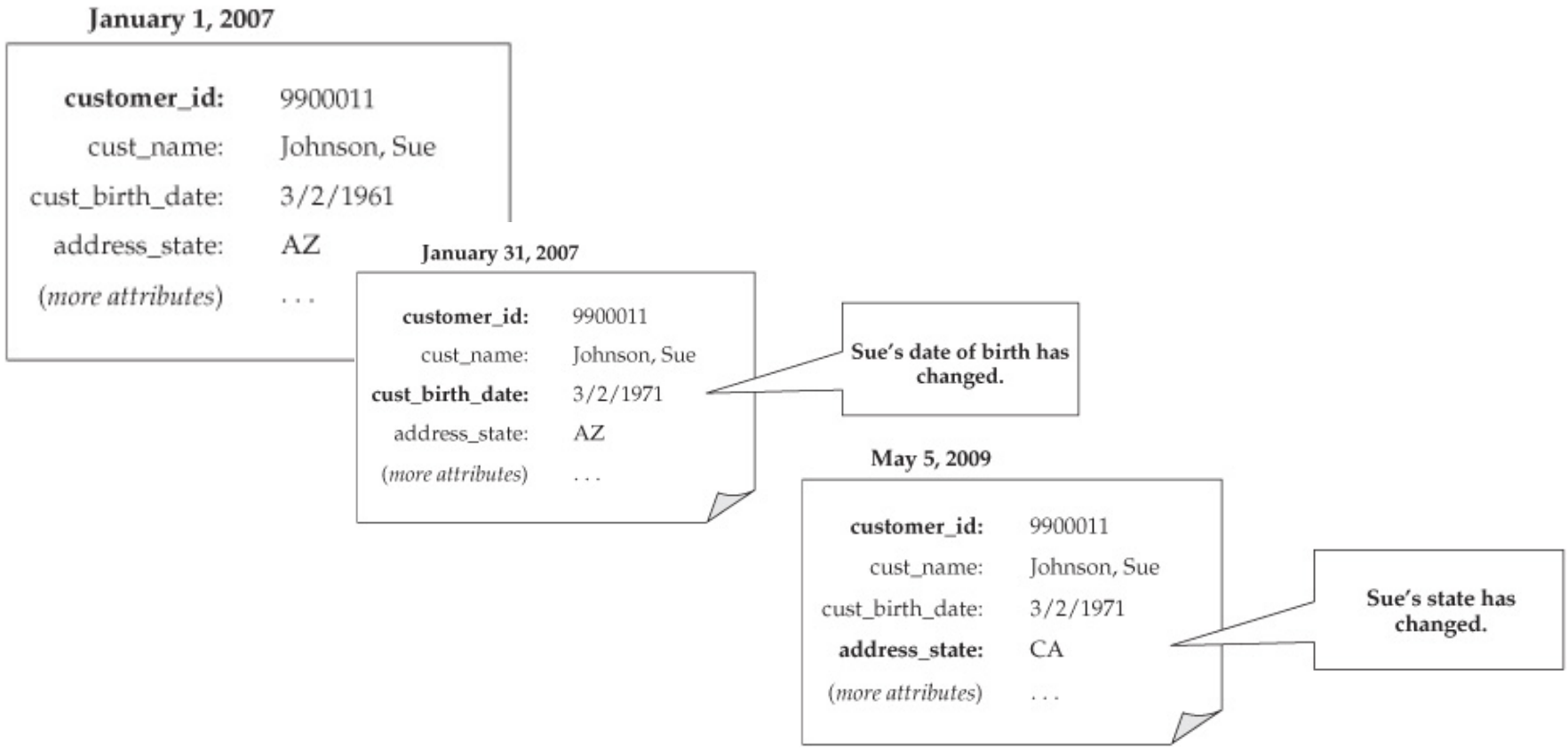
Example:

The business objective is to create a data model that can store and report number of burgers and fries sold from a specific McDonalds outlet per day.

What if a product's name is changed ?

What if a store's name is changed ?




January 1, 2007
customer_id: 9900011
cust_name: Johnson, Sue
cust_birth_date: 3/2/1961
address_state: AZ
(more attributes) . . .
January 31, 2007
customer_id: 9900011
cust_name: Johnson, Sue
cust_birth_date: 3/2/1971
address_state: AZ
(more attributes) . . .
Sue's date of birth has changed.
May 5, 2009
customer_id: 9900011
cust_name: Johnson, Sue
cust_birth_date: 3/2/1971
address_state: CA
(more attributes) . . .
Sue's state has changed.

Retrospection of the entity's existence:

- permanent retrospection

  (once set always the same)

- real retrospection

  (new versions are separate, requires additional time mark)

- false retrospection

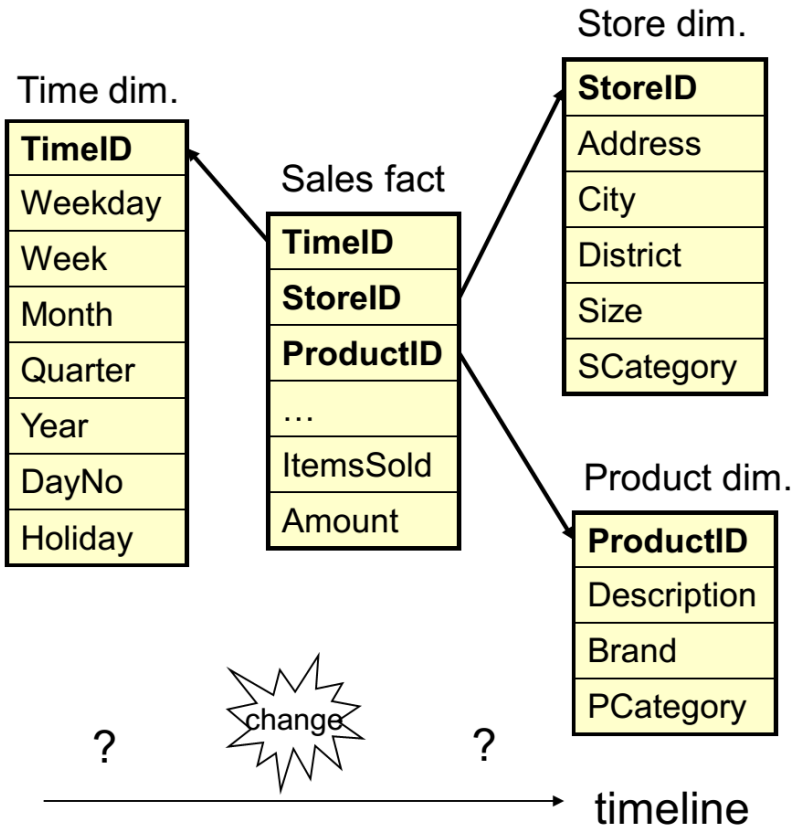
  (new versions replace old ones, there is no access to previous values)

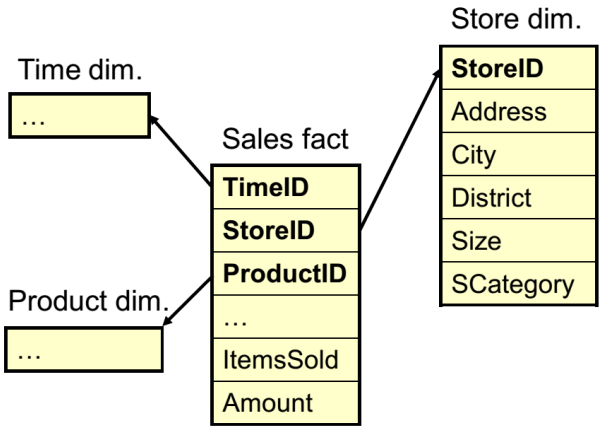
Yet some information is static

- Does not change over time
  - *In terms of size and content, during data warehouse usage*
- Can have different origins
  - *from data source*
    - typically dictionary-like structures
  - *or are created solely for data warehouse*
- Typically Date dimension is such a dimension
  - *E.g. all dates from 1/1/1990 to 12/31/2100*
  - *Most are calculated members*

Permanent dimension

- a form of a lookup table (dictionary) in a data warehouse environment
- Once set never changes
  - *No new data*
  - *No changes to data*
- It means that the update of dimension is outside of the ETL process
  - Typically before the ETL process
- Possible specific examples:
  - *Date, Location*


Store dim.
StoreID
Address
City
District
Size
SCategory
Time dim.
TimeID
Weekday
Week
Month
Quarter
Year
DayNo
Holiday
Sales fact
TimeID
StoreID
ProductID
…
ItemsSold
Amount
Product dim.
ProductID
Description
Brand
PCategory
change
?
?
timeline



Sales fact table

StoreID	…	ItemsSold	…
001		2000	

Store dimension table

StoreID	…	Size	…
001		250	

Slowly changing dimensions

- dimensions that change slowly over time
  - *rather than changing on regular schedule, time-base*
- in DW there is a need to track changes in dimension attributes in order to report historical data
  - *implementing one of the SCD types should enable users assigning proper dimension's attribute value for given date*
- example of such dimensions could be:
  - *customer, geography, employee.*

Sales fact table

StoreID	…	ItemsSold	…
001		2000	

Store dimension table

StoreID	…	Size	…
001		250	

The size of a store expands

StoreID	…	ItemsSold	…
001		2000	

StoreID	…	Size	…
001		450	

There are many approaches how to deal with SCD.

The most popular are:

- Type 0 - The passive method
- Type 1 - Overwriting the old value
- Type 2 - Creating a new additional record

- Type 3 - Adding a new column
- Type 4 - Using historical table

- Hybrid
  - *Type 5 – 1 + 4 (in a way 4)*
  - *Type 6 - Combine approaches of types 1,2,3 (1+2+3=6)*
  - *Type 7 – a bit different approach*

Slowly changing dimensions

- Type 0
  - *The Type 0 method is passive*
    - no special action is performed upon dimensional changes
  - *Dimension data remains the same as it was first inserted*
    - the values remain the same forever
    - new values can be introduced
- Type 0 is appropriate for any attribute labeled “original”
  - *such as a customer's original credit score or a durable identifier.*
  - *It also applies to most attributes in a date dimension.*

Slowly changing dimensions

Type 1

This methodology overwrites old with new data

therefore does not track historical data

type 1 attributes always reflects the most recent assignment

Is easy to implement

is often use for data which changes are caused by processing corrections

e.g. removal special characters, correcting spelling errors

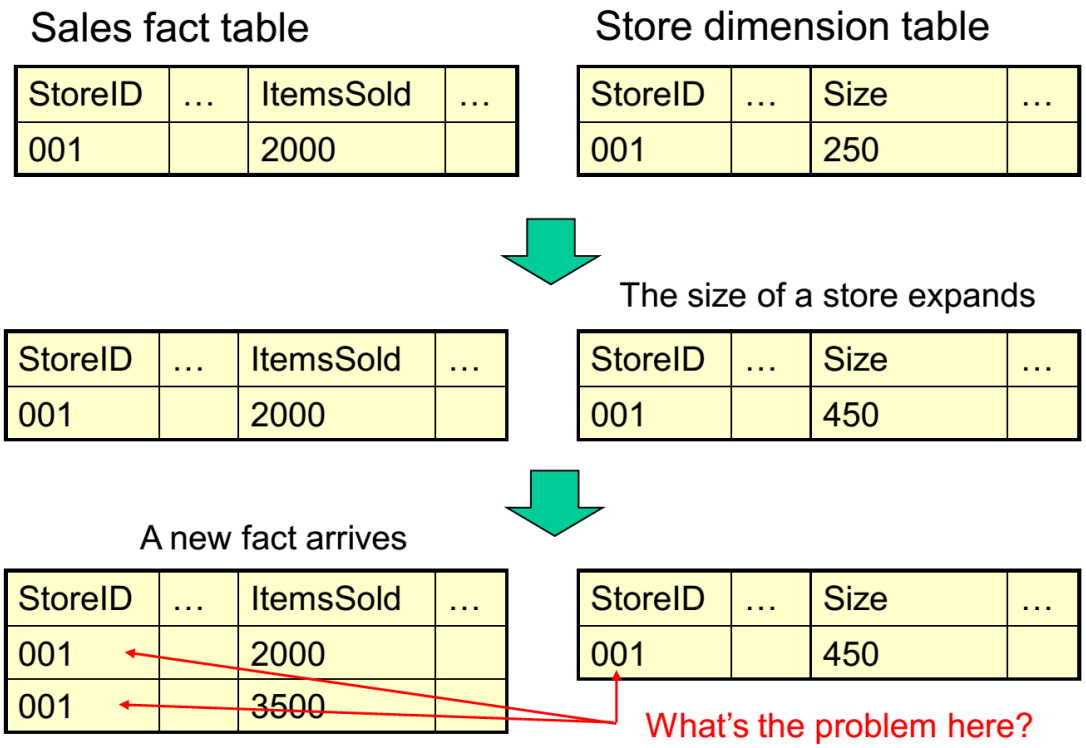
although this approach is easy to implement and does not create additional dimension rows, you must be careful that aggregate fact tables and OLAP cubes affected by this change are recomputed.

Slowly changing dimensions

Type 1 - overwrite

ID	Name	City
1	Nowak	Wroclaw

ID	Name	City
1	Nowak	Warsaw


Sales fact table
StoreID … ItemsSold …
001 2000
Store dimension table
StoreID … Size …
001 250
The size of a store expands
StoreID … ItemsSold …
001 2000
StoreID … Size …
001 450
A new fact arrives
StoreID … ItemsSold …
001 2000
001 3500
StoreID … Size …
001 450
What's the problem here?

Slowly changing dimensions

Type 2

Tracks historical data by creating multiple records

changes add a new row in the dimension with the updated attribute values

This requires generalizing the primary key of the dimension beyond the natural key because there will potentially be multiple rows describing each member.

for a given natural key in the dimensional tables with separate surrogate keys and/or different version numbers.

When a new row is created for a dimension member

a new primary surrogate key is assigned and used as a foreign key in all fact tables from the moment of the update until a subsequent change creates a new dimension key and updated dimension row.

Unlimited history is preserved

Slowly changing dimensions

Type 2

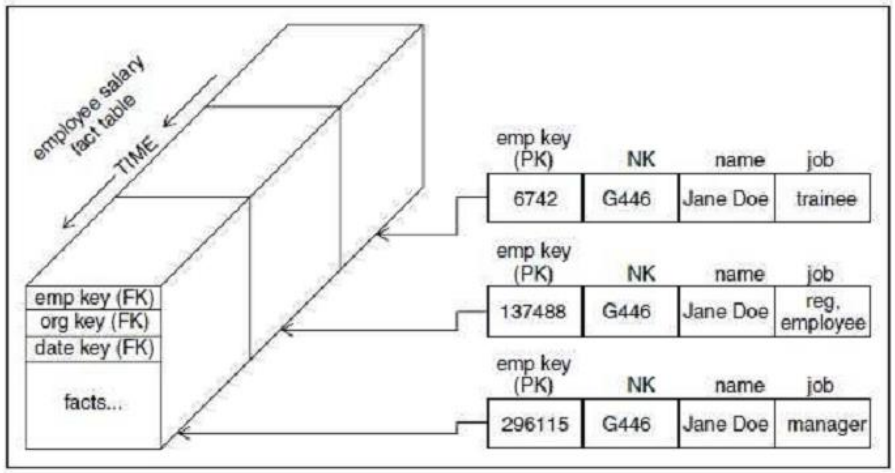
ID	Name	City
1	Nowak	Wroclaw

ID	Name	City	Current
1	Nowak	Wroclaw	0
2	Nowak	Warsaw	1

Slowly Changing Dimensions

The Type 2 SCD perfectly partitions history

each detailed version of a dimensional entity is correctly connected to the span of fact table records for which that version is exactly correct



Slowly changing dimensions

Type 2

For a given natural key in the dimensional tables with separate surrogate keys and/or different version numbers.

ID	Name	City
1	Nowak	Wroclaw

ID	NID	Name	City	Current
1	1	Nowak	Wroclaw	0
2	1	Nowak	Warsaw	1

Although the type 2 change preserves the historic context of facts, it does not preserve history in the dimension.

- given natural key has taken on multiple representations in the dimension
- we do not know when each of these representations was correct
  - *this information is only provided by way of a fact*

It may be clear to you that this problem is easily rectified by adding a date stamp to each version of "row"

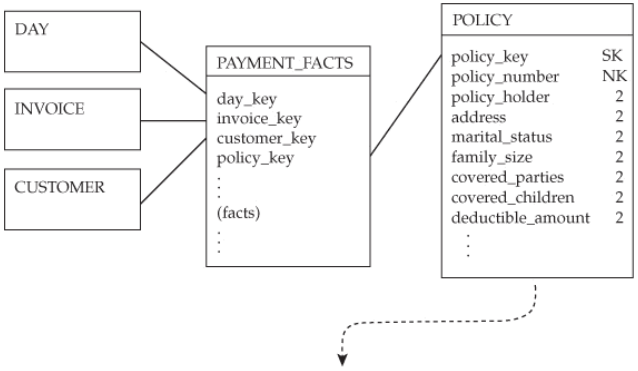
This technique allows the dimension to preserve both the history of facts and the history of dimensions.

Point-in-time status dimension

Time-stamped dimension

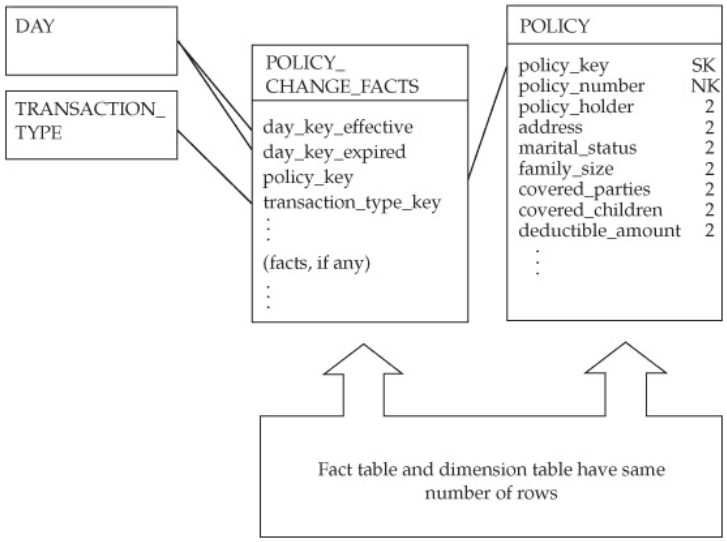
Slowly changing dimensions

Type 2 – point-in-time status dimension



POLICY

policy_ key	policy_ number	policy_ holder	address	marital_ status	family_ size	covered_ parties	covered_ children	deductible _amount
12882	40111	Smith, Hal	113 Random Rd.	Single	1	1	0	250
12911	40111	Smith, Hal	113 Random Rd.	Married	2	1	0	250
13400	40111	Smith, Hal	113 Random Rd.	Married	2	2	0	250
14779	40111	Smith, Hal	113 Random Rd.	Married	3	3	1	250
14922	40111	Smith, Hal	113 Random Rd.	Married	4	4	2	500



Slowly changing dimensions

Type 2 – time-stamped dimension

'effective date' and 'current indicator' columns are used in this method

only one record with current indicator set to 'Y'

'effective date' columns, i.e. start_date and end_date

the end_date for current record usually is set to value 9999-12-31

ID	Name	City
1	Nowak	Wroclaw

ID	NID	Name	City	Start	End
1	1	Nowak	Wroclaw	2014-10	2014-11
2	1	Nowak	Warsaw	2014-11	

Slowly changing dimensions

Type 2 – time-stamped dimension

- A time-stamped dimension adds non-overlapping effective and expiration dates for each row,
- Allows each type 2 change to be localized to a particular point in time.
  - *Effective date can be used to order a transaction history;*
  - *Effective and expiration dates allow filtering for a specific point in time;*
  - *Most_recent_version column can simplify filtering for current status.*

POLICY

policy_ key	policy_ number	policy_ holder	transaction_ type	effective_ date	expiration_ date	most_ recent_ version	marital_ status	family_ size	covered_ parties
12882	40111	Smith, Hal	New Policy	2/14/2005	2/11/2006	Expired	Single	1	1
12911	40111	Smith, Hal	Policy Change	2/12/2006	3/30/2006	Expired	Married	2	1
13400	40111	Smith, Hal	Policy Renewal	3/31/2006	12/19/2007	Expired	Married	2	2
14779	40111	Smith, Hal	Policy Change	12/20/2007	2/3/2008	Expired	Married	3	3
14922	40111	Smith, Hal	Policy Change	2/4/2008	12/31/9999	Current	Married	4	4

Use to order a change history

Use for point-in-time analysis across policies

Use to filter for current status

```
SELECT
 policy_holder,
 transaction_type,
 marital_status
   :
ORDER_BY
 effective_date
```

```
SELECT
   policy_holder,
   marital_status
     :
WHERE
 12/31/2006 >= effective_date AND
 12/31/2006 <= expiration_date
```

```
SELECT
   policy_holder,
   marital_status
     :
WHERE
 most_recent_row
     = "Current"
```

Slowly changing dimensions

Type 2

Introducing changes could be very expensive database operation

Introduces "abstract" entities

It allows for additional analysis (changes in dimension over time)

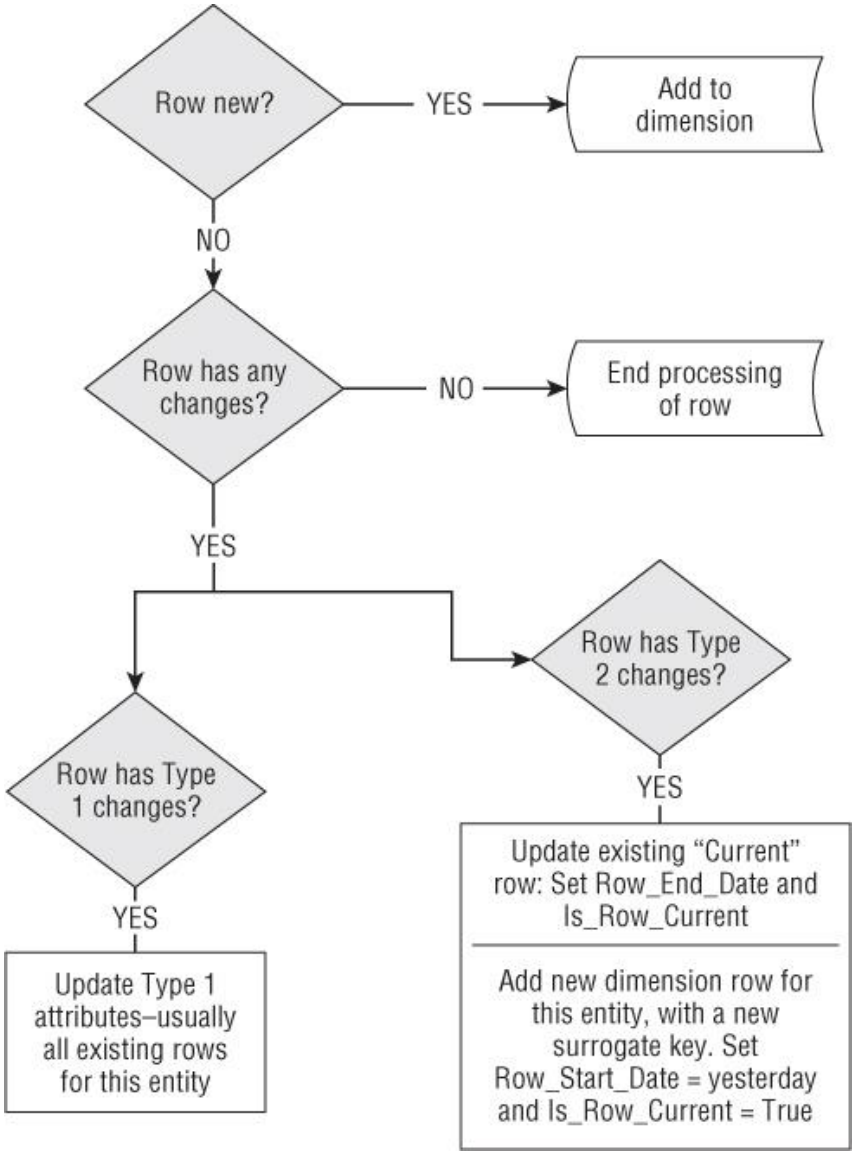
ID	Name	City
1	Nowak	Wroclaw

ID	NID	Name	City	Start	End
1	1	Nowak	Wroclaw	2014-10	2014-11
2	1	Nowak	Warsaw	2014-11	

The type 2 ETL process must also update the most recent surrogate key map table, assuming the ETL tool doesn't automatically handle this.

These little two-column tables are of immense importance when loading fact table data.

After you identify rows that have changes in type 2 attributes, you can generate a new surrogate key from the key sequence and update the surrogate key map table.



In most dimensional schemas

- changes to source data generate type 1 and type 2 changes
- occasionally, neither technique satisfies user requirements

Type 3

- when there is a need to analyse all facts, those recorded before and after the change, with either the old value or the new value
- neither type 1 nor type 2 does the job here
- preferred approach is to include two attributes for the changed data element
  - *one to carry the current value*
  - *one to carry the prior value*

Slowly changing dimensions

Type 3

Tracks changes using separate columns and preserves limited history.

limited to the number of columns designated for storing historical data.

usually only the current and previous value of dimension is kept

ID	Name	City
1	Nowak	Wroclaw

ID	Name	originalCity	Start	currentCity
1	Nowak	Wroclaw	2014-11	Warsaw

StoreID	…	ItemsSold	…
001		2000	

StoreID	…	DistrictOld	DistrictNew	…
001		37	37	

versions of an attribute

StoreID	…	ItemsSold	…
001		2000	

StoreID	…	DistrictOld	DistrictNew	…
001		37	73	

StoreID	…	ItemsSold	…
001		2000	
001		2100	

StoreID	…	DistrictOld	DistrictNew	…
001		37	73	

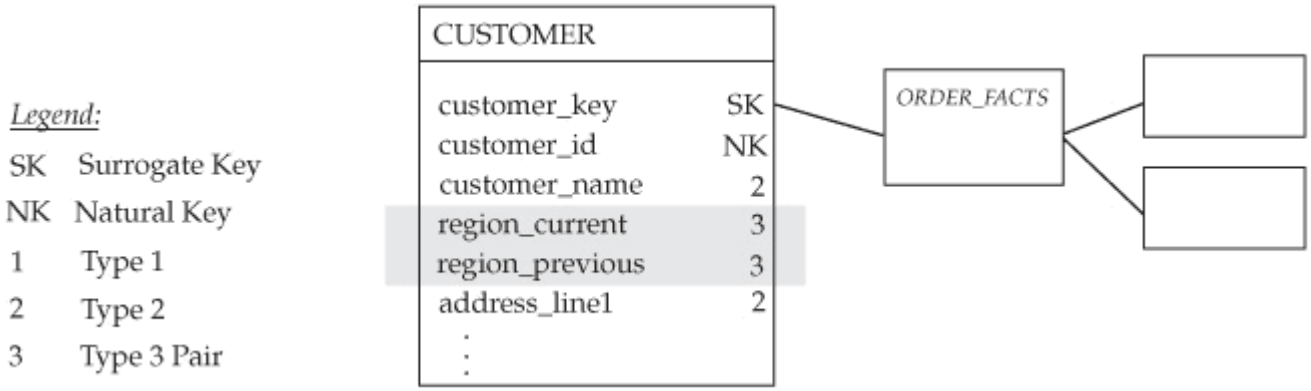
We cannot find out **when**
the district changed.

Type 3: Add New Attribute

is sometimes called an alternate reality.

A business user can group and filter fact data by either the current value or alternate reality.

This slowly changing dimension technique is used relatively infrequently.



Two seemingly contradictory requirements:

- The ability to analyse all facts, recorded before and after the change occurred, using the new value.
- The ability to analyse all facts, before and after the change occurred, using the old value.

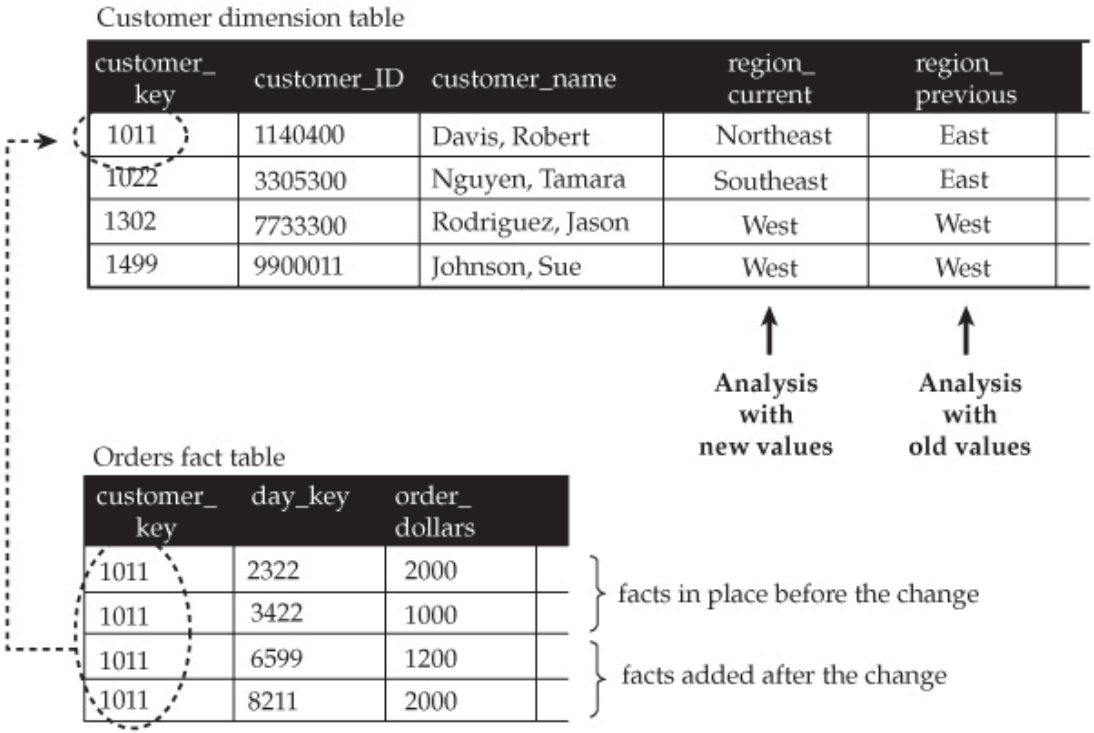
Type 3 change does not preserve the historic context of facts

- Each time a type 3 change occurs, the history is restated.

A type 3 change is not a one-time-only process.

- It can be repeated once, twice, or as many times as needed.

AFTER

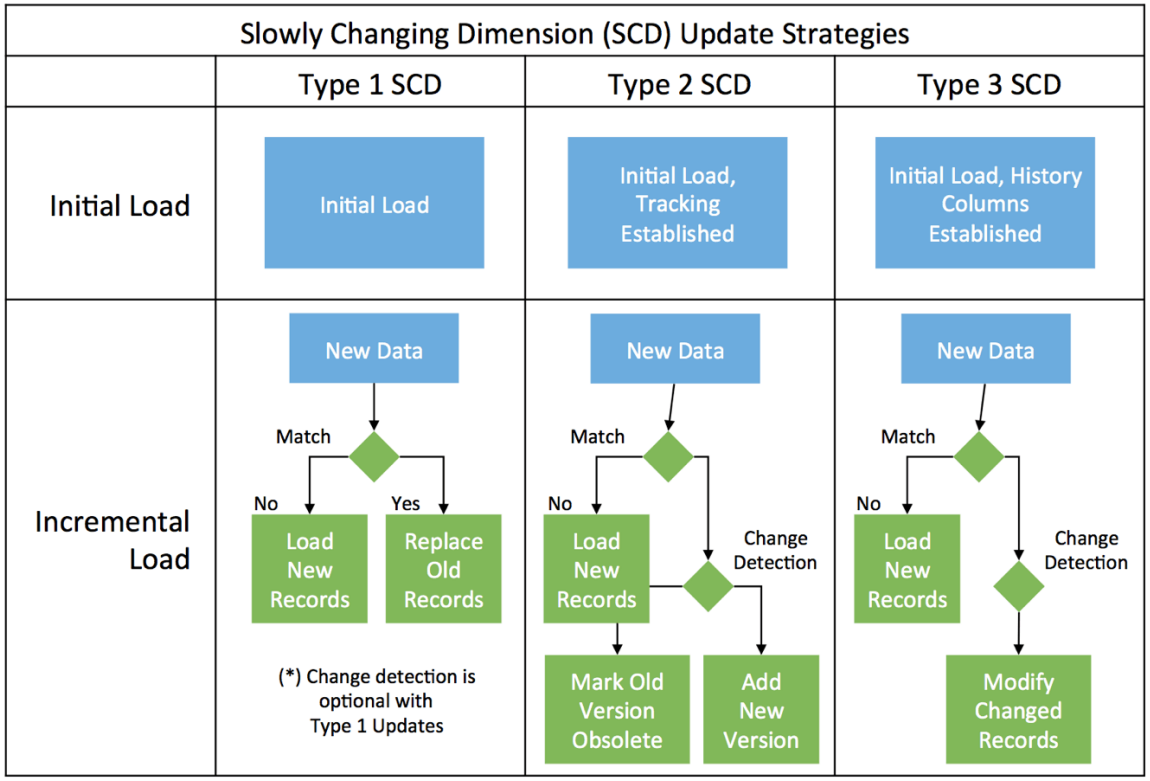

Customer dimension table
customer_ key
customer_ID
customer_name
region_ current
region_ previous
1011
1140400
Davis, Robert
Northeast
East
1022
3305300
Nguyen, Tamara
Southeast
East
1302
7733300
Rodriguez, Jason
West
West
1499
9900011
Johnson, Sue
West
West
Analysis with new values
Analysis with old values
Orders fact table
customer_ key
day_key
order_ dollars
1011
2322
2000
1011
3422
1000
1011
6599
1200
1011
8211
2000
facts in place before the change
facts added after the change

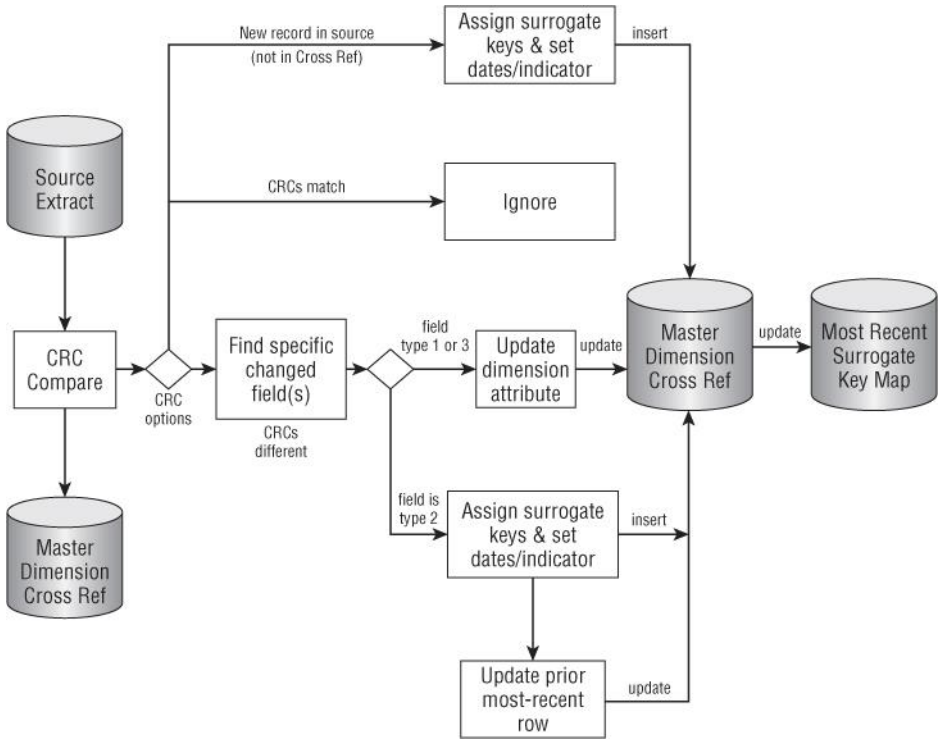
Slowly changing dimensions

Type 3

ID	Name	Rating	Rating-2013	Rating-2012	Start	
1	Nowak	1	1	2	2014-11	

ID	Name	Rating	Rating-1	Rating-2	Start	
1	Nowak	1	1	2	2014-11	


Slowly Changing Dimension (SCD) Update Strategies
Type 1 SCD
Type 2 SCD
Type 3 SCD
Initial Load
Initial Load
Initial Load, Tracking Established
Initial Load, History Columns Established
Incremental Load
New Data
Match
No
Yes
Load New Records
Replace Old Records
(*) Change detection is optional with Type 1 Updates
New Data
Match
No
Load New Records
Change Detection
Mark Old Version Obsolete
Add New Version
New Data
Match
No
Load New Records
Change Detection
Modify Changed Records


New record in source
(not in Cross Ref)
Assign surrogate keys & set dates/indicator
insert
Source Extract
CRCs match
Ignore
CRC Compare
CRC options
Find specific changed field(s)
CRCs different
field type 1 or 3
Update dimension attribute
update
Master Dimension Cross Ref
update
Most Recent Surrogate Key Map
field is type 2
Assign surrogate keys & set dates/indicator
insert
Master Dimension Cross Ref
Update prior most-recent row
update

CUSTOMER

customer_ key	company_ id	company_name _current	company_name _historic
1011	BB770	Apple Computer, Inc.	Apple Computer, Inc.

Name changes to Apple Inc.

CUSTOMER

customer_ key	company_ id	company_name _current	company_name _historic
1011	BB770	~~Apple Computer, Inc.~~ **Apple Inc.** 1	Apple Computer, Inc.
2822	BB770	Apple Inc.	Apple Inc.

2

Conflicting Requirements

Type 1 … and Type 2

Track name changes

Use current name

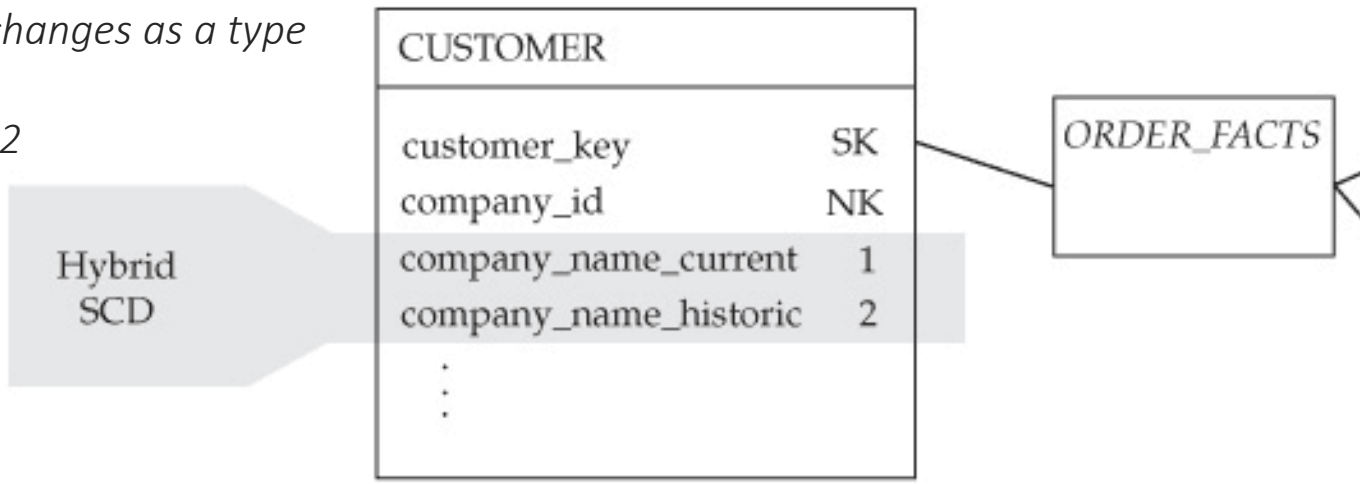
Type 3 is not adequate

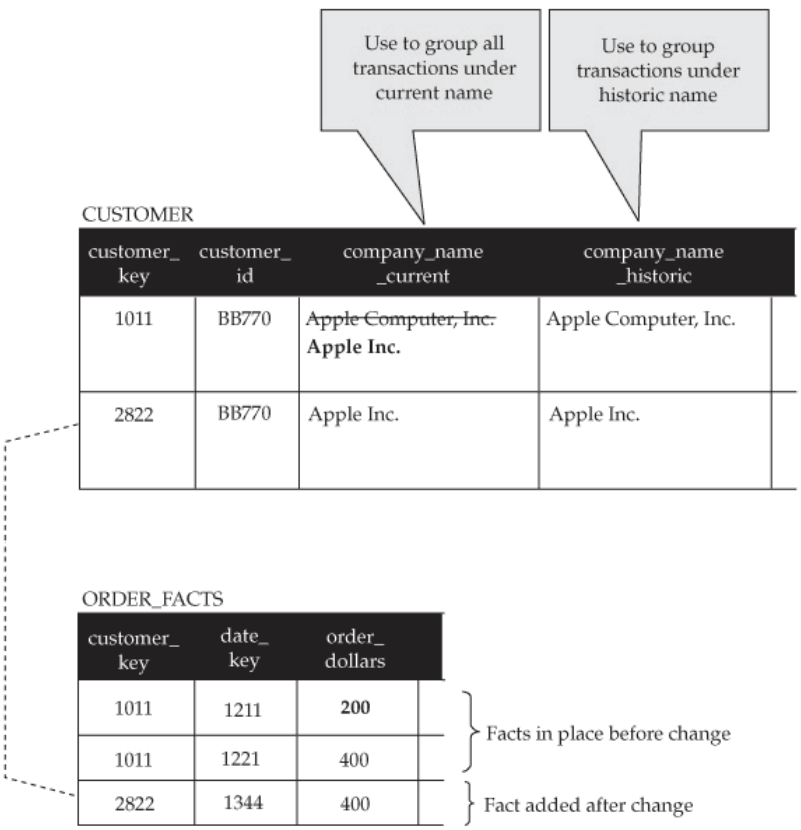
Type 1/2 hybrid

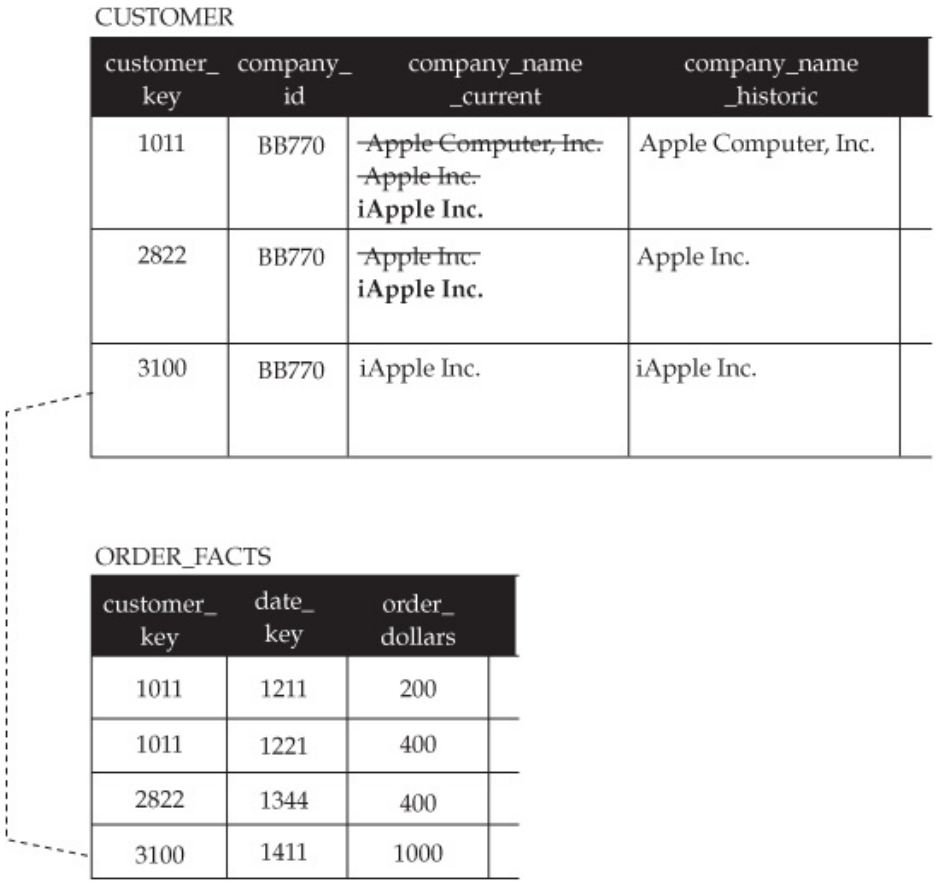
Provide a pair of attributes

one will react to changes as a type 1 response

the other as type 2




Use to group all transactions under current name
Use to group transactions under historic name
CUSTOMER
customer_ key
customer_ id
company_name _current
company_name _historic
1011
BB770
~~Apple Computer, Inc.~~ Apple Inc.
Apple Computer, Inc.
2822
BB770
Apple Inc.
Apple Inc.
ORDER_FACTS
customer_ key
date_ key
order_ dollars
1011
1211
200
1011
1221
400
2822
1344
400
Facts in place before change
Fact added after change


CUSTOMER
customer_ key
company_ id
company_name _current
company_name _historic
1011
BB770
~~Apple Computer, Inc.~~ ~~Apple Inc.~~ iApple Inc.
Apple Computer, Inc.
2822
BB770
~~Apple Inc.~~ iApple Inc.
Apple Inc.
3100
BB770
iApple Inc.
iApple Inc.
ORDER_FACTS
customer_ key
date_ key
order_ dollars
1011
1211
200
1011
1221
400
2822
1344
400
3100
1411
1000

Only implement hybrid changes when there are real analytic requirements.

- Operational reporting can be left to the operational system.

Slowly changing dimensions

Type 4

idea is to store all historical changes in a separate historical data table for each of the dimensions.

is useful when dimension attribute values are relatively volatile.

Slowly changing dimensions

Type 4

a separate table is used to track all attribute historical changes referred to as using "history tables“

one table keeps the current data

an additional table is used to keep a record of some or all changes

ID	Name	City
1	Nowak	Wroclaw

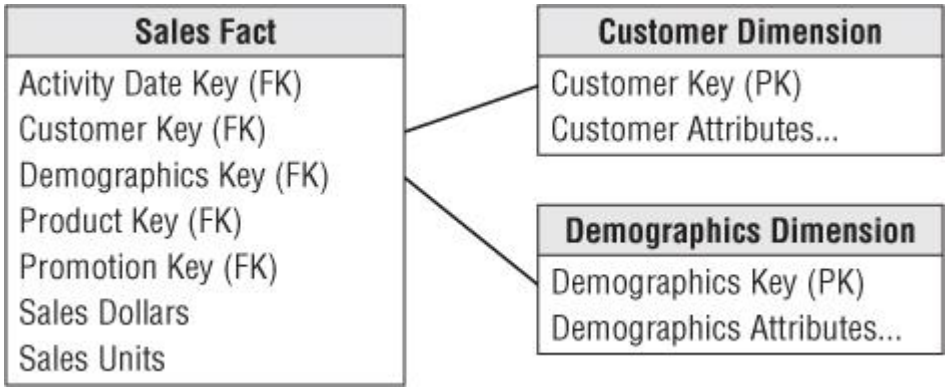
ID	Name	City
1	Nowak	Warsaw

ID	Name	originalCity	Start
1	Nowak	Wroclaw	2014-10

Type 4

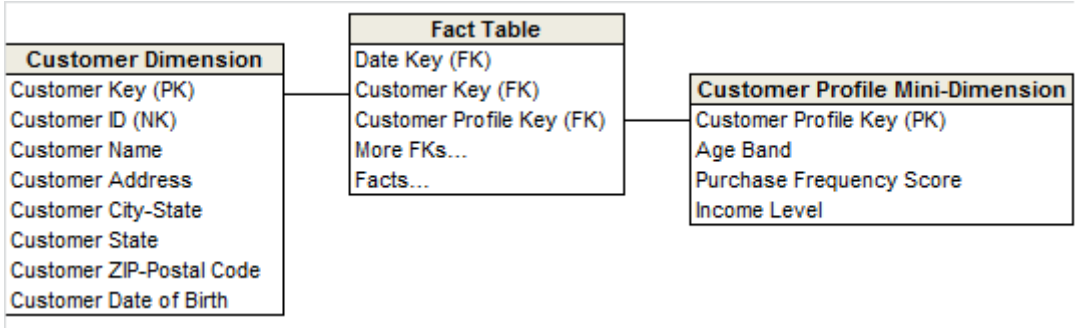
A surrogate key is assigned to each unique profile in the added dimension.

The surrogate keys of both the base dimension and added dimension profile are captured as foreign keys in the fact table



Type 4: Add Mini-Dimension

Technique is used when a group of attributes in a dimension change sufficiently rapidly so that they are split off to a mini-dimension.



Assume health clinic

Fact – individual patient's visit

Analysis1 – Which services are used most often by students?

Which types can we use for the customer dimension?

Analysis2 – What kind of services were used by customers who currently hold “gold” membership status?

Which type to use?

Fast changing dimension – mini-dimension approach

- If the attribute value changes often
  - *then a lot of rows from type 2*
- Imagine a health clinic
  - *Doctors can change lightning type in the room*
    - (warm, cold, natural)
  - *They do this multiple times a day*

Fast changing dimension – mini-dimension approach

Instead use additional dimension with all possible color modes

mini-dimension

In the health clinic example

- Additional dimension with all color modes
- Additional id in the fact table to the color dimension

Fast changing dimension – mini-dimension approach

In the health clinic example

Fact – individual patient's visit

Additional dimension with all color modes

Additional id in the fact table to the color dimension

Question

Can we analyse how the color changes in a room over time?

Fast changing dimension – mini-dimension approach

Consider a health clinic example

Patients rate doctors – on different aspects (1-10 scale)

Average monthly rate (for each doctor) is stored in the database

How to incorporate this data?

Fast changing dimension – mini-dimension approach

- If the attribute is numerical in nature
  - *maybe introduce a new fact table*
- Consider a health clinic example
  - *New fact table with the ratings*
  - *Conformed Date and Doctor dimensions*
- Alternatively we can use both
  - *Mini-dimension and new fact*

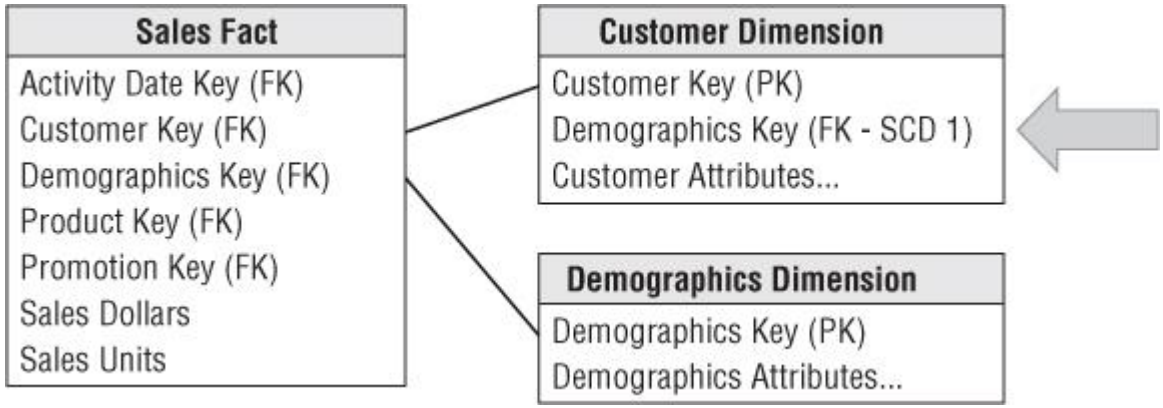
Slowly changing dimensions

Type 5

Combine approaches of types 1,4 (1 + mini-dimension = 5)

add Mini-Dimension and Type 1 Outrigger

Builds on the type 4 mini-dimension by embedding a "current profile" mini-dimension key in the base dimension that's overwritten as a type 1 attribute.

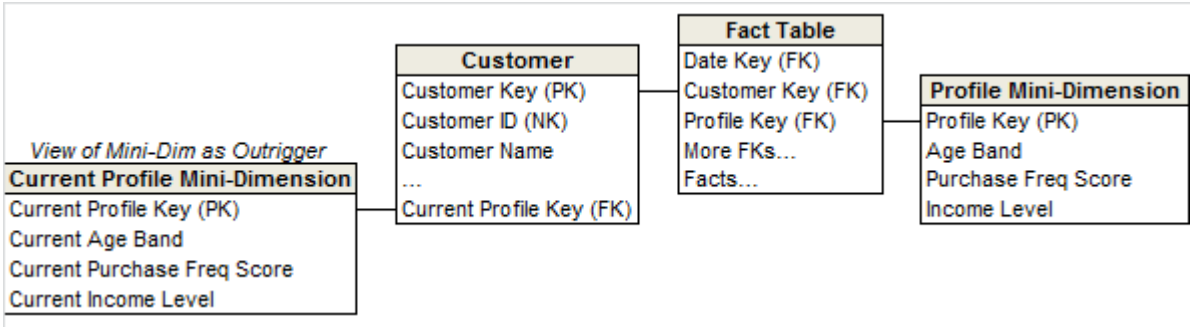


Slowly changing dimensions

Type 5

Logically, we typically represent the base dimension and current mini-dimension profile outrigger as a single table in the presentation layer.

The outrigger attributes should have distinct column names, like “Current Income Level,” to differentiate them from attributes in the mini-dimension linked to the fact table.

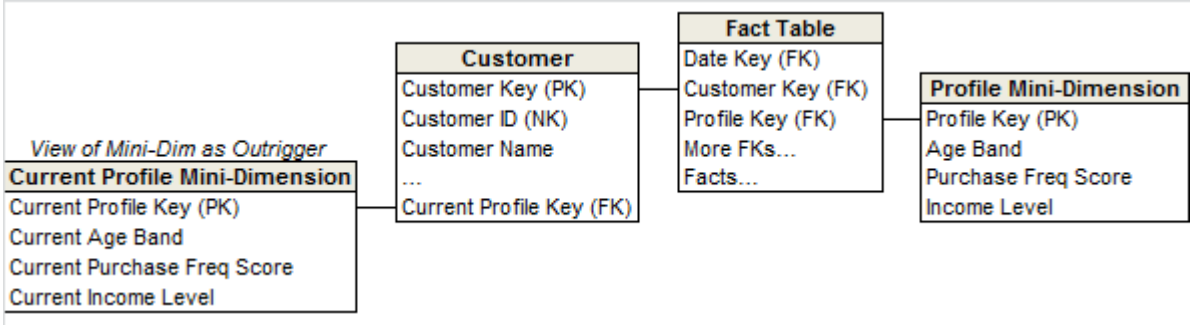


Slowly changing dimensions

Type 5

The ETL team must update/overwrite the type 1 mini-dimension reference whenever the current mini-dimension changes over time.

If the outrigger approach does not deliver satisfactory query performance, then the mini-dimension attributes could be physically embedded (and updated) in the base dimension



Slowly changing dimensions

Type 5

FID	CID	Key	…
1	100	3	
2	100	3	
3	101	1	

CID	NID	City	Key
100	8998829	Warsaw	1
101	8998829	Wroclaw	1

Key	A1	A2
1	1	0
2	0	1
3	1	1
4	0	0

Slowly changing dimensions

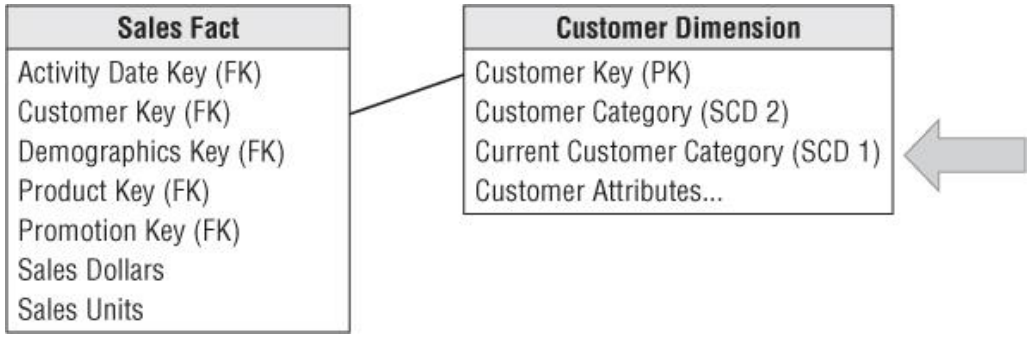
Type 6

Idea - add Type 1 Attributes to Type 2 Dimension

by also embedding current attributes in the dimension so that fact rows can be filtered or grouped by either the type 2 value in effect when the measurement occurred or the attribute's current value.

has an embedded attribute that is an alternate value of a normal type 2 attribute in the base dimension

Usually such an attribute is simply a type 3 alternative reality, but in this case the attribute is systematically overwritten whenever the attribute is updated.



Slowly changing dimensions

Type 6

Combine approaches of types 1,2,3 (1+2+3=6)

ID	Name	City
1	Nowak	Wroclaw

ID	Name	History_City	Current_City	Start	End	Current
1	Nowak	Warsaw	Wroclaw	2014-11	9999-12	Y
2	Nowak	Wroclaw	Wroclaw	2014-10	2014-11	N

Slowly changing dimensions

Type 6

Combine approaches of types 1,2,3 (1+2+3=6).

additional columns as:

current_type
  for keeping current value of the attribute.
  All history records for given item of attribute have the same current value.

historical_type
  for keeping historical value of the attribute.
  All history records for given item of attribute could have different values.

start_date
  for keeping start date of 'effective date' of attribute's history.

end_date
  for keeping end date of 'effective date' of attribute's history.

current_flag
  for keeping information about the most recent record.

Slowly changing dimensions

Type 6

Combine approaches of types 1,2,3 (1+2+3=6)

ID	Name	City
1	Nowak	Wroclaw

ID	Name	Current_City	City	History_City1	History_City2	Start	End	Current
0	Nowak	**Wroclaw**	Krakow			2014-10	2014-11	N
1	Nowak	**Wroclaw**	Warsaw	Krakow		2014-12	2015-01	N
2	Nowak	Wroclaw	Wroclaw	Warsaw	Krakow	2015-02	9999-12	Y

Original row in Product dimension:

Product Key	SKU (NK)	Product Description	Historic Department Name	Current Department Name	Row Effective Date	Row Expiration Date	Current Row Indicator
12345	ABC922-Z	IntelliKidz	Education	Education	2012-01-01	9999-12-31	Current

Rows in Product dimension following first department reassignment:

Product Key	SKU (NK)	Product Description	Historic Department Name	Current Department Name	Row Effective Date	Row Expiration Date	Current Row Indicator
12345	ABC922-Z	IntelliKidz	Education	Strategy	2012-01-01	2012-12-31	Expired
25984	ABC922-Z	IntelliKidz	Strategy	Strategy	2013-01-01	9999-12-31	Current

Rows in Product dimension following second department reassignment:

Product Key	SKU (NK)	Product Description	Historic Department Name	Current Department Name	Row Effective Date	Row Expiration Date	Current Row Indicator
12345	ABC922-Z	IntelliKidz	Education	Critical Thinking	2012-01-01	2012-12-31	Expired
25984	ABC922-Z	IntelliKidz	Strategy	Critical Thinking	2013-01-01	2013-02-03	Expired
31726	ABC922-Z	IntelliKidz	Critical Thinking	Critical Thinking	2013-02-04	9999-12-31	Current

Slowly changing dimensions

Type 7

Idea - Dual Type 1 and Type 2 Dimensions

A different mode of type 2

Add natural key (as additional) to fact table

Direct connection to current “instance” and general “instance”

Delivers the same functionality as type 6, but it’s accomplished via dual keys instead of physically overwriting the current attributes with type 6.

…	DIMID	DIMNID	…
	101	10	
	101	10	

DIMID	DIMNID	…
101	10	
102	10	

Slowly changing dimensions

Type 7

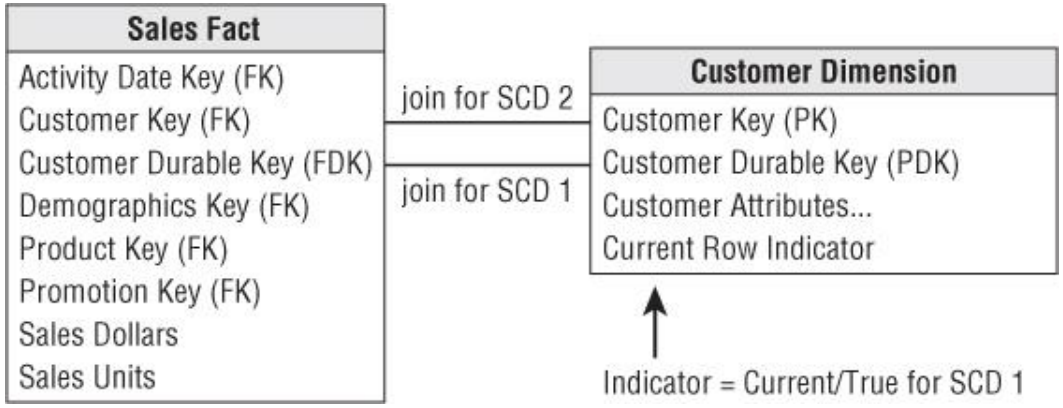
Idea - Dual Type 1 and Type 2 Dimensions

A different mode of type 2

Add natural key (as additional) to fact table

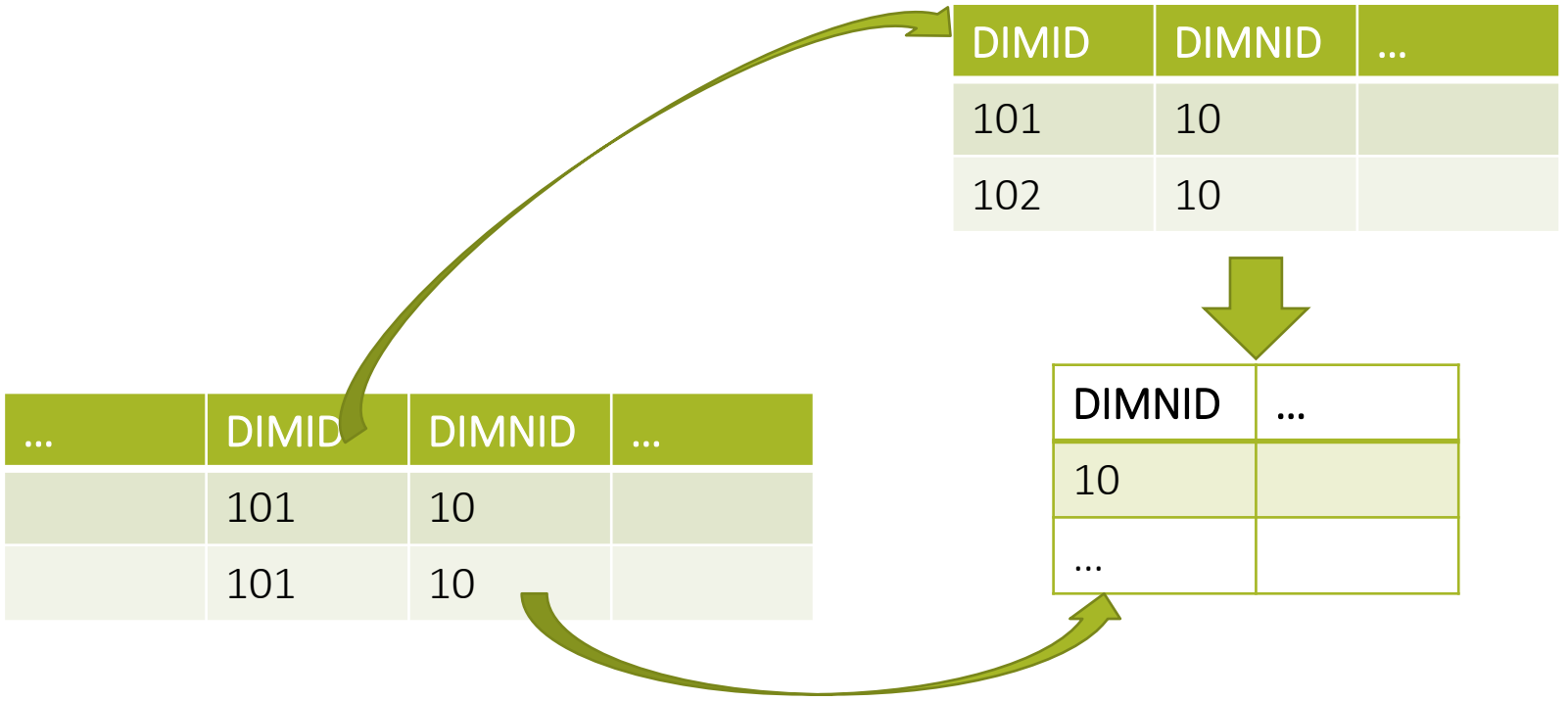
Direct connection to current “instance” and general “instance”

Delivers the same functionality as type 6, but it’s accomplished via dual keys instead of physically overwriting the current attributes with type 6.



Slowly changing dimensions

Type 7



Bibliography

SOURCES

Adamson C.

Star Schema The Complete Reference

McGraw-Hill, 2010

Kimball R., Ross M.

The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, Third Edition

John Wiley & Sons, Inc., 2013

Jensen C.S., Pedersen T.B., Thomsen C.,

Multidimensional Databases and Data Warehousing,

Morgan & Claypool Publishers series “Synthesis lectures on data management”, 2010

Inmon W.,

Building the Data Warehouse,

John Wiley & Sons, New York 2002

Claudia Imhoff, Nicholas Galemmo, Jonathan G. Geiger,

Mastering Data Warehouse Design - Relational and Dimensional Techniques,

Wiley Publishing, Inc., 2003